SIMATS ENGINEERING

CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.

Student Name	**P V GIRIDHAR**	**Register Number**	1972260022
Department / Section	CSE AI	**Academic Year / Semester**	2025-2026
Faculty Name	Dr. Krishna Kumar	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	**2-11-2026**	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

	Task Title	Max	Marks Awarded	Remarks
1	Design the Agent and the Search Algorithms	5		
2	Identify the Basic Operation	2		
3	Derive Time and Space Complexity Functions	6		
4	Best, Average and Worst-Case Analysis	4		
5	Asymptotic Analysis	3		
6	Formulate the Recurrence Relation	3		
7	Solve the Recurrence Relation	4		
8	Implement and Test the Algorithms	4		
9	Record Experimental Performance	3		
10	Compare Theory with Experiments	4		
11	Recommend and Defend the Final Design	4		
12	Re-evaluate and Adapt the Design (Day 4)	8		
	TOTAL	**50**		

OVERALL REMARKS:

Student Signature	**Faculty Signature**	**Program Director**

ASSIGNMENT 1 – CO1

ARTIFICIAL INTELLIGENCE – CSA17

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot **(Total Marks – 50)**

Duration: 1 Week

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach

- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

Answer:

PEAS	Robot
Performance	Correct delivery, least travel time, no collisions, low battery use.
Environment	Hospital corridors, junctions, lifts, ward doors, staff & trolleys.
Actuators	Wheels, lift/door interface, locked medicine tray.
Sensors	Lidar/ultrasonic, camera, wheel encoders, localisation, barcode reader.

View of AI:

Acting rationally. The robot is a goal-based problem-solving agent. It picks the move sequence that reaches the ward at least travel time, which is what a rational agent does.

This is the classic AI problem-solving agent setting: Newell, Simon and Shaw's General Problem Solver (1957) searched state spaces.

Formulation	Definition
States / Initial	Locations of the graph (N of them) / the pharmacy node.
Actions / transition	go(v) for each corridor (u,v); RESULT(u, go(v)) = v
Goal test / path cost	u = requested ward g / Sum of corridor times.

Preprocessing:

1. BFS (iterative, queue): Expand shallowest first.
   queue = [s]; while queue: u = pop-front; for v in adj[u]: if v new: parent[v] = u; if v == g return path.
2. DFS (iterative, stack): Expand deepest first.
   stack = [s]; while stack: u = pop; if u == g return path; for v in adj[u]: if v new: parent[v] = u; push(v)

3. UCS (iterative, priority queue)	Expand lowest cost g first; goal test when postponed.	Only one optimal for travel time.
4. IDDFS (recursive)	Run depth-limited DFS with limit 0, 1, 2, …	BFS-quality route with tiny memory;

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm
- Why it is appropriate for efficiency analysis
- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

Answer:

Algorithm	Where it occurs	How it grows with N
BFS/DFS	Inner loop over adj[u] for every expanded node.	Up to the no. of nodes seen, never more than N; in this hospital a const. fraction of N.
UCS	Same loop plus a heap push, O(log n) each.	Nodes with cost < C^*; at most N+2E pushes.
IDDFS	Every recursive call of DLS, repeated in each iteration.	About b^d per final pass plus repeated shallower pass;

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor *b*, shallowest-goal depth *d*, maximum depth *m*, optimal cost C^*, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

Answer:

BFS: Level i has b^i nodes. With goal test at generation, nodes generated = $1 + b + \ldots + b^d \Rightarrow (b^{d+1} - 1)/(b - 1) = O(b^d)$. With an explored set each node is generated once & each corridor examined twice, so also ≤ N and O(N+E).

DFS: Tree form visits up to depth m. Graph form ≤ N generations, O(N+E). Space: O(bm) in tree form.

UCS: There are at most N+2E heap pushes of O(log N) each: O((N+E) log N) time, O(N) space.

IDDFS: Pass L generates $(b^{L+1} - 1)/(b - 1)$ nodes, so $T = \Sigma_{L=0}^{d} (b^{L+1} - 1)/(b - 1) = (b^{d+2} - b - (d+1)(b - 1))/(b - 1)^2 \geq b^d \Rightarrow O(b^d)$.

Equivalently a node at depth j is regenerated in (d − j + 1) passes, so $T = \Sigma (d - j + 1) \cdot b^j$. Space is just the current path, O(d). Total $\Theta(N+E) + O(b^d)$.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)
- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

Answer:

Algorithm	Best case	Average case	Worst case
BFS	Goal = start or a neighbor; deg(s) nodes.	Goal at depth ≈ $\log_b N$; the last layers hold most nodes.	Goal in the deepest layer or unreachable; all N nodes.
DFS	Goal on the first branch followed.	Long wandering dive first.	All N nodes; it ignores cost, so cheap-edge detours do not matter.
UCS	Goal is the cheapest neighbor.	Goal at depth $d \approx 2.25 \cdot b^d$; duplicates are regenerated.	Unreachable goal: it would have to enumerate every simple path.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

Answer:

- **Growth:** Here $d \approx \log_b N$, so $b^d \approx N$ & all four are Θ(N) in time. Measured (log-log) slopes of nodes generated from 1000 to 1,000,000: BFS 0.96, DFS 1.04, UCS 0.99, IDDFS 0.93.
- **Memory limits first:** At 1,000,000 locations BFS keeps ≈ 211,686 frontier entries & UCS ≈ 317,552. IDDFS only 14.
- **Caveat:** Real buildings have long corridors, so d can far exceed log N. Then $b^d \gg N$ & IDDFS becomes hopeless while BFS/UCS stay bounded by N+E.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

Evidence Expected: A correct recurrence with clear identification of its components and base case.

Answer:

Recursive algorithm: depth-limited search DLS(u, g, l). Reading its code: the test u = g and the test l = 0 cost constant time; then the loop makes one recursive call DLS(v, g, l−1).

Component	In my algorithm
Problem size	Remaining depth limit l
Subproblem size	l − 1 (subtract by one)
No. of calls	b (one per child)
Work outside calls	Constant c (goal test, limit test, path-set lookup)
Base case	l = 0; goal test, return CUTOFF; $T(0) = c_0$

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

Answer:

The recurrence shrinks the problem by 1 rather than dividing it, so the Master Theorem does not apply. It is linear with a constant term, so the repeated substitution works.

Meaning: Extra level of depth multiplies the work by b = 3, while memory stays Θ(l): time-hungry, memory-light.

Iterative deepening: $I(d) = \Sigma_{L=0}^{d} (b^{L+1} - 1)/(b - 1) = (b^{d+2} - b - (d+1)(b - 1))/(b - 1)^2 \Rightarrow \Theta(b^d)$. For b = 3, d =

12 this is 1,195,735 calls against 797,161 for one pass of depth 12, only about 50% more (ratio → b/(b − 1)).

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

Answer:

Test	Expected	Actual
Reachable goal (all sizes).	Valid route from all four; BFS/IDDFS fewest corridors.	All 144 of 144 runs valid. BFS fewest-corridors 9/9 at every size.
Start = goal	Route [s], 1 node generated.	All four return [s] with 1 generated at all sizes.
Unreachable ward.	BFS/DFS/UCS fail after exploring everything.	BFS, DFS generated exactly N.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

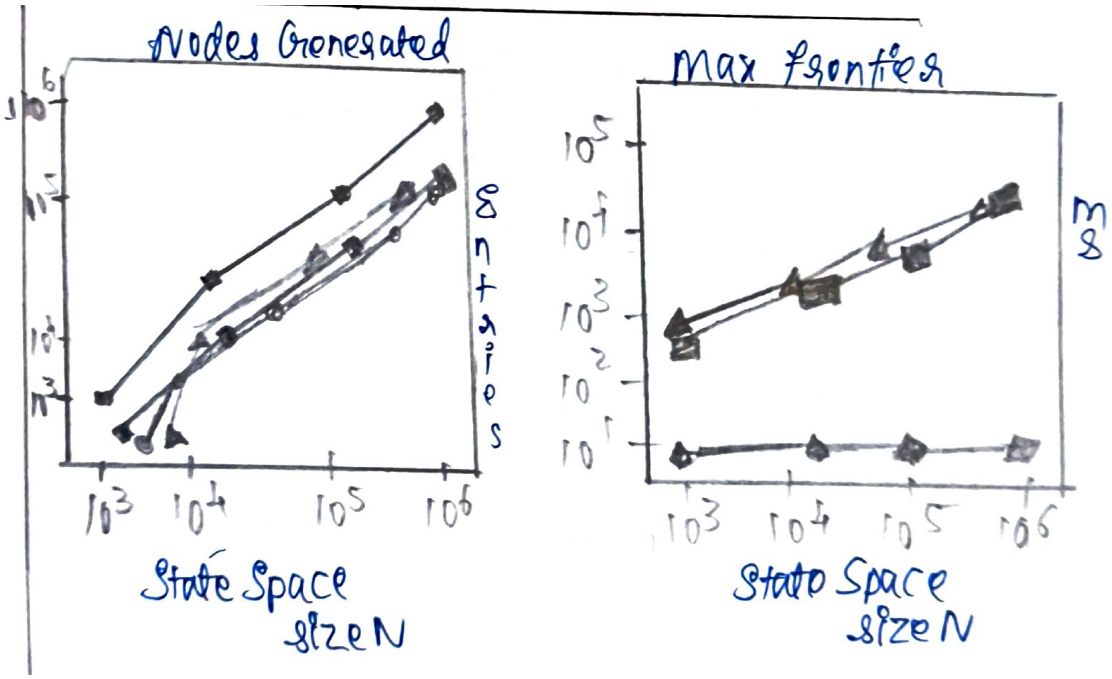
- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

Answer:

Nodes Generated and Max Frontier vs State Space size N (log scale):



TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance.

Answer:

- **Agreement:** Nodes generated grew linearly as predicted when $b^d \approx N$. Solution depth rose about 2 corridors per ×10 in N (6.1→8.7→10.7→12.7) against ($\log_3 10 = 2.1$). BFS frontiers stayed ≈ 24%, 26%, 21%, 21% of N while IDDFS recursion depth grew like the depth.
- **IDDFS formula:** With b = 3 & each query's own depth, observed counts were 0.74, 0.54, 0.40, 0.52 of the Task 7 prediction.
- **Best scaler:** BFS, fewest nodes among the explored-set methods & fastest. IDDFS wins on memory.
- **Differences & reasons:**

  (1) Duplicates: BFS discards them, IDDFS regenerates them, so IDDFS used 4.2× BFS instead of the theoretical 1.5×.

  (2) The goal is usually found before its layer is finished, so BFS generated 42–56% of N, not 100%.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

Answer:

- **Objective is travel time:** BFS minimizes corridors & was cost-optimal in only 5, 4, 3, 1 of 9 queries per size. UCS in all 9. At 1M the mean cost was 38.9.
- **Others rejected:** DFS routes averaged 107,968 corridors for goals about 13 away.
- **Efficiency:** Theory O((N+E) log N) time & O(N) memory; observed 77%, 68%, 81%, 70% of N generated ≈ 1400 ms per query.
- **Trade offs:** About 5× slower than BFS & a frontier of ≈ 317,552 entries. If the robot cannot hold that, plan on the hospital server or use depth-capped IDDFS where d is small.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

A. Impact (2):

The start is no longer one node but a set of k possible junctions, so the known-start & fully-observable assumptions fail.

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for *k* possible locations, and is the original design acceptable?

B. Original Design:

BFS/UCS cannot be used as they are (a state is no longer a location). They can run over belief states. There are up to 2^n beliefs, but an action never enlarges a belief.

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

C. Decision: MODIFY

Keep graph search with a frontier & explored set but change the state to a belief state. No sensor: BFS over belief finds one fixed move sequence that works for all k states.

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for k = 2, 4, 8, with and without a local sensor), with code and output as proof

MARK DISTRIBUTION – ASSIGNMENT 1

Task	Marks
Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**